

Lab of Algorithm

Assignment 8

Implementation and Use-Case Comparison of Four Shortest Path Algorithms

Student ID: 411321344

Name: Wilkend AUGUSTE

Course: Lab of Algorithm

Date: June 10, 2026



National Dong Hwa University

Department of Computer Science and Information Engineering

1. Problem Definition

The shortest path problem asks: given a graph $G = (V, E)$, find the minimum-cost path between vertices. This is one of the most fundamental problems in graph theory, with real-world applications in GPS navigation, network routing, game AI pathfinding, and logistics planning.

The problem takes different forms depending on graph properties:

- Unweighted graph: every edge costs 1 (or the same). Shortest path = fewest edges. BFS solves this optimally.
- Weighted non-negative graph: edges have costs ≥ 0 . Dijkstra is the standard single-source solution.
- Weighted with negative edges: at least one edge cost < 0 . Bellman-Ford handles this and detects negative cycles.
- All-pairs: we need the shortest distance between every pair (i, j) . Floyd-Warshall is the classical DP approach.

This assignment implements all four algorithms (BFS, Dijkstra, Bellman-Ford, Floyd-Warshall) plus the A* bonus, compares them across five designed test cases, and analyzes when each algorithm should or should not be chosen.

2. Algorithm Explanations

2.1 BFS Shortest Path

BFS (Breadth-First Search) expands the graph level by level starting from a source vertex. Because every edge costs the same (or the graph is unweighted), the first time BFS reaches a vertex v , it must have taken the fewest possible edges, there is no shorter route because no earlier level reached it. The key data structures are a FIFO queue for the frontier and a `dist []` array initialized to -1. When vertex u is dequeued, every unvisited neighbor v gets `dist[v] = dist[u] + 1` and is enqueued. A `parent []` array reconstructs the exact path.

Time complexity is $O(V + E)$ since every vertex and edge is processed at most once. BFS is only correct for unweighted graphs. On a weighted graph, it ignores costs and can report a 1-edge path costing 100 as shorter than a 3-edge path costing 3.

2.2 Dijkstra Algorithm

Dijkstra solves single-source shortest paths when all edge weights are non-negative. The core insight is the greedy choice property: once vertex u is extracted with the smallest known distance, that distance is final and cannot be improved later, no future edge can decrease it because all weights are ≥ 0 . A min-heap (priority queue) of (distance, vertex) pairs efficiently selects the next minimum.

When processing vertex u , every outgoing edge $u \rightarrow v$ with weight w is relaxed: if `dist[u] + w < dist[v]`, update `dist[v]` and push the new entry to the heap. Outdated heap entries where the

popped distance exceeds the current dist [] are simply skipped. Time complexity with a binary heap is $O((V + E) \log V)$.

Critical limitation: if a negative-weight edge exists, a vertex already confirmed as "minimum" could later be improved through that edge, but Dijkstra never revisits confirmed vertices, producing silently wrong answers.

2.3 Bellman-Ford Algorithm

Bellman-Ford is a single-source shortest path algorithm that correctly handles negative-weight edges. Instead of a greedy approach, it uses repeated relaxation: all edges are relaxed $|V| - 1$ times. The reasoning is that in a graph without negative cycles, any shortest path visits at most $|V| - 1$ edges, so after $|V| - 1$ full passes every distance is guaranteed to be correct.

After $|V| - 1$ passes, one extra pass checks for further improvements. If any distance still decreases, a negative-weight cycle reachable from the source exists, distances would keep decreasing forever, so they are undefined. The implementation uses a flat edge list (not an adjacency list), which differs from Dijkstra. Time complexity is $O(V \times E)$.

2.4 Floyd-Warshall Algorithm

Floyd-Warshall computes all-pairs shortest paths using dynamic programming in a single $O(V^3)$ pass. It maintains a $V \times V$ matrix $\text{dist}[i][j]$ and iterates over every possible intermediate vertex k : for each pair (i, j) , if routing through k is shorter ($\text{dist}[i][k] + \text{dist}[k][j] < \text{dist}[i][j]$), the matrix is updated. After all V iterations of the outer loop, the matrix holds correct shortest distances between every pair.

A next $[][]$ matrix records the successor on the shortest path, enabling full path reconstruction. If any diagonal entry $\text{dist}[i][i]$ becomes negative, vertex i is on a negative cycle. Floyd-Warshall is elegant and compact, but its cubic cost makes it impractical for large graphs ($V > \sim 500$).

2.5 A* Search Algorithm (Bonus)

A* extends Dijkstra by adding a heuristic function $h(n)$ that estimates the remaining cost to the target. The evaluation function is:

$$f(n) = g(n) + h(n)$$

- $g(n)$: exact cost from start to node n (same as Dijkstra's $\text{dist}[]$).
- $h(n)$: estimated cost from node n to the target. Must be admissible (never overestimate).
- $f(n)$: priority used by the min-heap. A* expands the node with the smallest $f(n)$ first.

For a 2D grid with 4-directional movement, Manhattan distance $h(n) = |\text{row}_n - \text{row}_t| + |\text{col}_n - \text{col}_t|$ is a perfect admissible heuristic. When $h(n) = 0$ for all n , A* degenerates exactly to Dijkstra. A well-chosen heuristic dramatically reduces the number of nodes expanded by guiding the search toward the goal.

3. Program Design

3.1 File Structure

Each algorithm is implemented in a separate C++ source file:

- bfs_shortest_path.cpp.
BFS on undirected unweighted graphs, outputs min-edge distances and path.
- dijkstra.cpp.
Dijkstra with min-heap priority queue on non-negative weighted graphs.
- bellman_ford.cpp.
Bellman-Ford on directed graphs, handles negative edges and detects negative cycles.
- floyd_warshall.cpp.
Floyd-Warshall all-pairs, with path reconstruction via next [] [] matrix.
- astar.cpp.
A* on a 2D grid map with Manhattan heuristic and Dijkstra comparison.

3.2 Data Structures Used

- BFS: vector<vector<int>> adjacency list, queue<int>, int dist [] and parent [] arrays.
- Dijkstra: vector<vector<pair<int, int>>> weighted adjacency list, priority_queue (min-heap of (dist, vertex)), dist [] and parent [].
- Bellman-Ford: vector<Edge> flat edge list (struct {int u, v, w}), dist [] and parent [] arrays.
- Floyd-Warshall: 2D vectors dist[V][V] and next[V][V] for distances and path reconstruction.
- A*: 2D grid vector, g_cost[rows][cols], priority_queue<tuple<int, int, int>> for (f, row, col), parent[rows][cols].

3.3 Input / Output Format

All programs read from standard input. First line: "V E". Next E lines: edge definitions (u v for unweighted, u v w for weighted). Final line: "source target". Each program outputs all distances from source and reconstructs the path to target. Bellman-Ford additionally reports a negative cycle warning when detected.

4. Test Cases and Outputs

BFS on Unweighted Undirected Graph

Graph: 8 vertices, 9 undirected edges (no weights). Source: 0, Target: 7. All edges represent equal-cost connections.

Observation: BFS correctly computes the minimum number of edges to reach each node. The distances match exactly the level in the BFS tree. Path $0 \rightarrow 2 \rightarrow 5 \rightarrow 7$ uses 3 edges, which is the minimum possible — no shorter path (by edge count) exists.

```
Distances: 0(0) 1(1) 2(1) 3(2) 4(2) 5(2) 6(3) 7(3)
Path from 0 to 7: 0 -> 2 -> 5 -> 7    Total edges: 3
```

Dijkstra on Non-Negative Weighted Graph

Graph: 6 vertices, 8 undirected weighted edges (all weights ≥ 0). Source: 0, Target: 5.

Observation: The priority queue correctly processes nodes in distance order. The shortest path $0 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 5$ (total cost 10) beats the direct $0 \rightarrow 1$ path (cost $4+5+2=11$). This confirms Dijkstra finds the weight-optimal route, not just the fewest-edge route.

```
Distances: 0(0) 1(3) 2(1) 3(8) 4(11) 5(10)
Path from 0 to 5: 0 -> 2 -> 1 -> 3 -> 5    Cost: 10
```

Bellman-Ford with Negative Edges (No Cycle)

Graph: 5 vertices, 6 directed edges. Edge $2 \rightarrow 1$ has weight -2 (negative). Source: 0, Target: 4.

Observation: Bellman-Ford correctly handles the negative edge. The shortest path $0 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 4$ (cost 8) uses the negative edge $2 \rightarrow 1$ (cost -2) to reduce the total below the direct $0 \rightarrow 1$ route (cost 4). No negative cycle is detected. This test demonstrates a case where Dijkstra would give the wrong answer (it would not update node 1 via the negative edge).

```
No negative-weight cycle detected.
Distances: 0(0) 1(3) 2(5) 3(6) 4(8)
Path from 0 to 4: 0 -> 2 -> 1 -> 3 -> 4    Cost: 8
```

Floyd-Warshall All-Pairs Shortest Distances

Graph: 5 vertices, 7 undirected weighted edges. All weights positive.

Observation: The all-pairs distance matrix is correctly computed in a single $O(V^3)$ pass. The matrix is symmetric (undirected graph) and diagonal is all zeros. Path $0 \rightarrow 4$ is $0 \rightarrow 1 \rightarrow 3 \rightarrow 4$ (cost 8), which was verified manually. Floyd-Warshall provides all 25 pairwise distances in one run.

```
Distance matrix (rows/cols = vertices 0-4) :
      [0]  [1]  [2]  [3]  [4]
[0]    0    3    5    4    8
[1]    3    0    2    1    5
[2]    5    2    0    2    5
[3]    4    1    2    0    4
[4]    8    5    5    4    0
Path from 0 to 4: 0 -> 1 -> 3 -> 4    Cost: 8
```

Negative Cycle Detection

Graph: 4 vertices, 4 directed edges. Cycle $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ has total weight $3 + (-8) + 2 = -3$, which is a negative cycle reachable from source 0.

Observation: After $|V|-1 = 3$ relaxation passes, the extra 4th pass still finds an improvement on node 2. Bellman-Ford correctly reports the negative cycle and terminates without producing undefined distances. This confirms the algorithm works as a reliable negative-cycle detector.

```
[WARNING] Negative-weight cycle detected reachable from source 0!
Shortest distances are not well-defined (can be -infinity).
A node affected by the cycle: 2
```

A* on 2D Grid (Bonus)

Grid: 7×9 with wall obstacles. Start: (0, 0), Target: (6, 8). Heuristic: Manhattan distance $h(n) = |\text{row}_n - \text{row}_t| + |\text{col}_n - \text{col}_t|$.

Observation: A* finds the optimal path of length 14. Both A* and Dijkstra expanded 41 nodes on this maze. In open spaces without obstacles, A* typically expands significantly fewer nodes because the heuristic guides search toward the goal. The grid visualization clearly shows the path threading along the top row then down the rightmost column around the walls.

```
Path length (cost): 14    |    A* nodes expanded: 41
Grid: S * * * * * * * *
      . # # # . # # # *
      . . . # . . . # *    ... T
```

5. Algorithm Comparison

The table below summarizes each algorithm's applicable graph type, ability to handle negative weights, scope, time complexity, and key strengths and limitations.

Algorithm	Graph Type	Neg. Weights?	Scope	Time Complexity	Strengths & Limitations
BFS	Unweighted or equal-weight (directed or undirected)	No	Single-source	$O(V + E)$	Optimal for min-edge paths; fails on weighted graphs with different costs
Dijkstra	Weighted, all non-negative (directed or undirected)	No (non-negative only)	Single-source	$O((V+E) \log V)$	Fast & practical; incorrect if any negative weight exists
Bellman-Ford	Weighted, may have negative edges (directed)	Yes — also detects negative cycles	Single-source	$O(V \times E)$	Handles negative edges; much slower than Dijkstra for dense graphs
Floyd-Warshall	Any weighted graph (directed or undirected)	Yes — detects via negative diagonal	All-pairs	$O(V^3)$	Simple all-pairs DP; impractical for large V (e.g. >500 nodes)

Key observations from the comparison:

- BFS is the fastest at $O(V+E)$ but is only correct for unweighted (or equal-weight) graphs. It is the right choice when edges represent uniform steps.
- Dijkstra is the practical standard for single-source problems with non-negative weights. The priority queue keeps it efficient even for dense graphs.
- Bellman-Ford is slower ($O(V \times E)$) but handles negative edges and detects negative cycles. It is the only correct single-source algorithm in these situations.
- Floyd-Warshall computes all-pairs shortest paths in one elegant triple-loop, but $O(V^3)$ limits it to smaller graphs (typically $V \leq 500$).

6. Analysis of Incorrect Use Cases

BFS on a Weighted Graph

If BFS is applied to a weighted graph, it ignores costs and reports the path with the fewest edges, not the minimum weight. For example: nodes 0, 1, 2 with edges $0 \rightarrow 1$ (weight 100) and $0 \rightarrow 2 \rightarrow 1$ (weights 1+1). BFS reports path $0 \rightarrow 1$ (1 edge, cost 100) as shorter than $0 \rightarrow 2 \rightarrow 1$ (2 edges, cost 2). The result is wrong by a factor of 50. BFS should never be used when edges have different costs.

Dijkstra on a Graph with Negative Weights

Dijkstra's greedy guarantee states: once vertex u is extracted with distance d , that distance is final. This holds only if all weights are ≥ 0 , because no future edge can make d smaller. With a negative edge, a later path could reduce an already-confirmed distance, but Dijkstra never revisits confirmed vertices, it produces a silently wrong answer with no error.

Example: nodes 0, 1, 2 with edges $0 \rightarrow 1$ (1), $0 \rightarrow 2$ (2), $2 \rightarrow 1$ (-2). Dijkstra confirms $\text{dist}[1] = 1$ (via direct edge). When it later processes $0 \rightarrow 2$ ($\text{dist}=2$), it tries to relax $2 \rightarrow 1$: $\text{dist}[2] + (-2) = 0 < 1$, but vertex 1 is already confirmed and is not updated. The correct answer is 0, but Dijkstra returns 1.

Is Floyd-Warshall Always Best for Single-Source?

No. Floyd-Warshall runs in $O(V^3)$ regardless of the query. For single-source problems, Dijkstra runs in $O((V+E) \log V)$, which is much faster for sparse graphs. On a graph with 1,000 vertices and 2,000 edges: Dijkstra takes roughly 20,000 operations; Floyd-Warshall takes 1,000,000,000. Floyd-Warshall is only justified when many all-pairs queries are needed, and the graph is small enough for cubic cost to be acceptable.

Adjacency List vs. Matrix for Sparse Graphs

For graphs with many vertices but few edges, an adjacency list is strongly preferred. An adjacency matrix requires $O(V^2)$ space regardless of edge count for 10,000 vertices that is 100 million entries. An adjacency list uses only $O(V + E)$, proportional to what actually exists. Iterating over neighbors is $O(\text{degree})$ with a list vs. $O(V)$ with a matrix, making BFS and Dijkstra faster on sparse graphs.

7. Conclusion and Personal Reflection

This assignment showed that there is no single "best" shortest path algorithm, the right choice depends entirely on the graph's properties and the type of query needed.

- When edges are unweighted or equal-cost, BFS is the simplest and most efficient solution.
- When all weights are non-negative and only one source is needed, Dijkstra with a priority queue is the standard choice.
- When negative edges exist (without a negative cycle), Bellman-Ford is the only correct single-source algorithm; its extra relaxation pass also serves as a reliable negative-cycle detector.
- When shortest distances between every pair of vertices are needed on a small graph, Floyd-Warshall's compact triple-loop DP is intuitive and complete.
- A* extends Dijkstra with a heuristic to direct the search toward the target, making it significantly more efficient in practice when a good admissible heuristic exists.

The key skill this assignment developed is not memorizing algorithms but learning to match each algorithm to the assumptions it requires. BFS assumes uniformity; Dijkstra assumes non-negativity; Bellman-Ford assumes finite paths (or detects when they are not); Floyd-Warshall assumes all-pairs is needed at once. Violating those assumptions produces wrong results, understanding why is what separates correctly using an algorithm from just running it.

Implementing and testing all four algorithms on carefully designed cases, including the negative-cycle trap and a grid maze for A* gave concrete evidence of both their power and their limits.